

AMAbot & Git basics

💡 Dagens fokus

Analysér gængse chatbot UIs, byg jeres eget, og brug Git til at tracke udviklingen.



Dagens program

8:15	15	🎓	Intro & agenda
8:30	30	🎓	Tag fotos til studiekort
9:00	20	💡	<u>Opsamling på mind maps og placering i semestre</u>
9:20	5	🎓	Det' mig
9:25	10	🎓	<u>Intro til chatbot projektet</u>
9:35	10	🎓	<u>Demo: En hurtig analyse i DevTools</u>
9:45	20	🔗	<u>Undersøg andre chatbots</u>
10:05	15	💡	Præsentation af chatbot analyser
10:20	20	⌚	Pause
10:40	20	🎓	<u>Git basics — commits</u>
11:00	20	🎓	<u>Git staging area og commit beskeder</u>
11:20	10	💡	<u>Commit via GitHub Desktop, VS Code og Terminal</u>

1. Opsamling på mind maps og placering i semestre

Læg en plan på tavlen

→ Baseret på jeres mind maps fra sidst skitserer vi et potentielt forløb på tavlen for de næste to semestre.

2. Det' mig



Lecturer
Erhvervsakademi Aarhus | Business Academy Aarhus · Full-time
Aug 2021 - Present · 5 yrs 1 mo
Aarhus, Central Denmark Region, Denmark
Teaching the spectrum from basic JavaScript to advanced frontend development.

Lead Frontend Developer
Wise Home
Apr 2018 - Jul 2021 · 3 yrs 4 mos
Ry, Denmark

Building the frontend of internal tools, customer-facing interfaces and public websites in React and React Native, with a dedication to performance, accessibility and simplicity.

Wise Home develops "smart home" solutions for apartment buildings, giving both residents and property administrators real-time insights into relative humidity, ventilation levels, heat and water consumption and other data points via IoT sensors that communicate wirelessly to a central server.

Frontend Developer
Gaest.com
Sep 2016 - Mar 2018 · 1 yr 7 mos
Greater Aarhus Area

Building the Gaest.com frontend in React JS (against Rails-based JSON API and Elixir-based GraphQL backends).

Mining for data insights in PostgreSQL and building internal data visualization dashboards in Metabase.

Launching rocket ships to infinity and beyond — an amazing startup ride with a stellar team.

Gaest.com makes it easy to quickly find, book and rent out venues for meetings, workshops and conferences. Over 3,000 unique venues in 8 countries, and new countries launching globally throughout 2019. In January 2019 Gaest was acquired by Airbnb.

Frontend Developer
Illumi
Mar 2011 - Aug 2016 · 5 yrs 6 mos

Building flexible websites, mostly on the Umbraco CMS platform — XSLT ftw.

Responsive web design with a dedicated focus on frontend performance — built and launched a fully responsive ecommerce site in the fall of 2011 which might well have been (one of) the first of its kind in the country.

Illumi is an Aarhus-based consultancy and web agency specializing in solid and dependable web solutions and bespoke software integrations, built by a group of very talented, friendly and honest people.

"Our First Umbraco 7 Build" - Session at Umbraco Codegarden 2014
A presentation I gave with my Illumi colleague Kenn Jacobsen at the 2014 Umbraco developer conference, detailing our experiences and workflow around the newest ...

Project Lead
Klean A/S
Oct 2009 - Feb 2011 · 1 yr 5 mos

Scrum master on agile web development projects.

Conducted functional, automated and usability testing of websites.

Migrated source control from several diverse systems to Git and GitHub back before it was cool (or at least not very well supported on Windows).

3. Intro til chatbot projektet

I bygger en komplet chatbot-applikation (individuelt) over den første måned som den røde tråd gennem hele forløbet.

Teknisk progression

- Start: Server-side rendering (SSR)
- Derefter: EJS templating → session-baseret tilstand → moderne API-udvikling
- Slut: En SPA (Single-Page Application) der snakker JSON med en API backend

Læring

- Grundig forståelse af client-server arkitektur
- Forstå hvordan webudvikling har udviklet sig gennem de seneste årtier

Slutmål

- Navigér igennem jeres Git commits og forklar de forskellige iterationer af jeres chatbot
- Vis funktionaliteten i browser og DevTools
- Forklar udvalgt kode linje-for-linje

4. Demo: En hurtig analyse i DevTools

Lær at bruge DevTools til at analysere en webseite og forstå dens opbygning.

💡 Hvad skal I lede efter:

- **Requests og ressourcer:** Antal HTTP-requests og den samlede “vægt” af ressourcer (CSS, JS, billeder osv.).
- **Frameworks og teknologier:** Identifikation af teknologier som React, Angular, eller server-side rendering.

📄 Trin-for-trin:

- 01 Åbn DevTools i din browser (højreklik → “Inspicer”).
- 02 Gå til fanen “Netværk” og genindlæs siden.
- 03 Notér antal requests og den samlede størrelse af ressourcer (nederst i netværksoversigten).
- 04 Skift til fanen “Kilder” eller “Elements” for at finde spor af frameworks.

5. Undersøg andre chatbots

👥 Grupper ⌚ 20 min

I skal i grupper analysere eksisterende chatbot interfaces ved hjælp af browserens DevTools — for at få indsigt i, hvordan professionelle chatbots er bygget, og hvilke teknologier og designvalg der ligger bag.

Del evt arbejdet op så I arbejder parvis og sammenlign noter efterfølgende. Vær klar til at præsentere jeres fund for klassen.

📄 Trin-for-trin analyse:

- 01 Vælg en chatbot fra tavlen og åbn den i browseren.
- 02 Brug DevTools (“Inspicer”) til at åbne “Netværk”-fanen og genindlæs siden.
- 03 Notér antal HTTP-requests og samlet ressourceforbrug (MB/kB).
- 04 Undersøg “Elements”-fanen for at identificere hvilket frontend framework og evt CSS framework, der er brugt.
- 05 Identificér de primære UI-komponenter (fx chatbesked, inputfelt, send-knap).
- 06 Læg mærke til interaktionsmønstre: Hvordan starter man en ny chat? Hvordan vises beskeder?

💬 Diskutér i gruppen:

- Hvilke designvalg virker gode? Hvad kunne forbedres?
- Virker det som en rimelig “vægt” af ressourcer for den funktionalitet, der tilbydes? Er det mere eller mindre end I forventede?

6. Versionskontrol: Save states til din kode

Tænk på Git som et save system

I et spil:

- Save point før en boss fight
- Gem forskellige endings
- Load tidligere save hvis I dør
- Flere save slots for at eksperimentere
- Checkpoint system holder historik

I Git:

- Commit før store refactors
- Branches til forskellige features
- Checkout til tidligere commit hvis noget går galt
- Flere branches til at teste idéer
- Commit history viser hele udviklingen



Hvorfor er det brugbart?

- **Frygtløs udvikling:** I kan altid load en tidligere “save”
- **Eksperimentér frit:** Lav en branch, test vilde idéer, discard hvis det ikke virker
- **Samarbejd uden kaos:** Alle kan arbejde parallelt uden at overskrive hinandens arbejde
- **Se udviklingen:** Som et speedrun — se hele projektets progression

7. Git basics – commits

💡 Hvad er et commit?

- **Snapshot af kode:** Et commit kan konceptualiseres som et øjebliksbillede af alle filer på et bestemt tidspunkt
- **Metadata:** Indeholder forfatter, tidsstempel og commit-besked
- **What's my name:** Hver commit får en SHA-1 hash (fx `a1b2c3d`) som unik ID
- **Who's my daddy:** Et commit peger på det forrige commit (undtagen det første)

📦 Hvorfor er commits vigtige?

- **Historik og sporbarhed:** Se præcis hvad der er ændret og hvornår
- **Kollaboration:** Flere udviklere kan arbejde på samme projekt uden konflikter
- **Fejlfinding:** Gå tilbage til tidligere versioner hvis noget går galt
- **Dokumentation:** Commit-beskeder forklarer *hvorfor* ændringer blev lavet



7.1 Directed Acyclic Graph (DAG) strukturen

Git organiserer commits i en DAG-struktur, som gør det muligt at spore udvikling og håndtere branches.

💡 Hvad betyder DAG?

- **Directed:** Commits peger én retning (bagud i tid via parent-referencer)
- **Acyclic:** Ingen loops - man kan ikke gå i cirkel gennem commit-historikken
- **Graph:** Netværk af forbundne knuder (commits) og linjer (parent-child relationer)

🔄 I jeres projekter:

- Hver ny feature eller bugfix bliver et nyt commit
- I kan eksperimentere på branches uden at ødelægge koden
- I kan altid “spole tilbage” til tidligere iterationer, enten via branches, tags eller commit hashes



7.2 Git staging area og commit beskeder

Git har et unikt “staging area”-koncept mellem jeres aktuelle filer på disk og det permanente repository, som giver præcis kontrol over hvad der committes.

💡 De tre områder i Git:

- **Working Directory:** Jeres faktiske filer, som I kan redigere
- **Staging Area (Index):** En “ventezone” for ændringer klar til commit
- **Repository (.git):** Permanent opbevaring af alle commits og historik

🕒 Hvorfor staging area?

- **Selektive commits:** Commit kun specifikke dele af jeres arbejde
- **Gennemgang før commit:** Tjek præcis hvad der kommer med i committet
- **Atomare ændringer:** Hver commit fokuserer på én logisk ændring

📄 Git-kommandoer og flow:

- `git status` viser tilstanden af alle tre områder
- `git add <fil>` flytter ændringer fra working directory til staging area
- `git commit -m "besked"` laver et permanent commit af det stagede indhold

7.3 Gode commit-beskeder og best practices

Veldokumenterede commits gør det lettere for andre (og jer selv senere) at forstå projektets udvikling.

Commit-besked struktur:

Kort, præcis opsummering (under 50 tegn)

Uddybende forklaring hvis nødvendigt. Forklar HVORFOR ændringen blev lavet, ikke bare hvad der blev ændret.

- Bullet points med specifikke ændringer
- Referencer til issues: "Fixes #42"

Dårlige commit beskeder

"fixed stuff", "updates", "lol"

Bedre commit beskeder

"Make chat message component responsive",
"fix: Handle empty user input in chat form"

Best practices for jeres projekter:

- **Commit ofte:** Små, logiske ændringer er bedre end store, komplekse commits
- **Test før commit:** Sørg for koden virker før I committer
- **Konsistent format:** Brug imperativ form ("Add feature" ikke "Added feature")

7.4 Commit via GitHub Desktop, VS Code og Terminal

Lær de forskellige måder at arbejde med Git gennem hands-on demonstration af samme workflow i tre populære værktøjer.

💡 Hvorfor flere værktøjer?

→ Forskellige præferencer:

Nogle foretrækker visuelle interfaces, andre kommandolinjen

→ Kontekst-afhængig:

Terminal til scripting, GUI til kompleks merge-konfliktløsning

→ Læringsværdi:

Forståelse af underliggende Git-koncepter på tværs af værktøjer

📄 Vi gennemgår samme workflow:

- 01 Tilføj en ny fil til jeres chatbot projekt
- 02 Stage ændringerne for commit
- 03 Skriv en commit-besked og commit
- 04 Push til GitHub repository

7.5 Vælg jeres foretrukne workflow

Der er ingen “rigtig” måde at bruge Git på - vælg det værktøj der passer til jeres arbejdsstil og erfaring.

Diskutér i par

- Hvilket værktøj føltes mest intuitivt for jer?
- Hvilke situationer kan I forestille jer at bruge hvert værktøj til?

Mangle udviklere kombinerer værktøjer

- **VS Code til daglige commits:** Hurtige commits under udvikling
- **Terminal til avancerede operationer:** Rebase, cherry-picking, og automatisering

I jeres chatbot-projekter

- **Opnå fortrolighed:** Brug samme værktøj indtil I er trygge ved Git-koncepterne
- **Commit tidligt og ofte:** Små, logiske ændringer er bedst — uanset værktøj
- **Eksperimentér:** Prøv forskellige værktøjer indtil I finder jeres favorit